

Hunter & Craftsman 部署配置及问题处理说明

面向项目接手、部署实施与运维使用

版本日期：2026-07-15

1. 文档目的与适用范围

本文用于说明 Hunter 与 Craftsman 当前版本的推荐部署方式、关键配置项、启动顺序、Google Play internal 测试轨道发布前提，以及常见问题的处理方法。适用于项目接手人员、部署实施人员和日常运维人员。

当前版本建议按“Craftsman 服务 + Dashboard + 后台 Worker”与“Hunter 触发器”两部分理解。用户侧主要通过 Dashboard 观察需求发现、代码生成、质量评分和发布状态；Hunter 负责触发自动发现、自动生成和自动发布流程。

2. 系统部署结构

组件	职责	部署建议	是否需要外网入站
Craftsman	提供 Dashboard、HTTP API、任务编排、后台 Worker	常驻服务	通常需要
Hunter	触发需求发现、代码生成、发布流程	命令行触发或定时任务	不需要
Google Play API	接收元数据、AAB、轨道提交	云侧能力	不适用
DeepSeek / 模型接口	支持分析与代码生成	云侧能力	不适用

推荐的线上访问路径如下：

浏览器 -> Nginx 80/443 -> 127.0.0.1:8791 -> Craftsman Dashboard + API
Hunter -> http://127.0.0.1:8791 -> Craftsman API
Craftsman -> DeepSeek / Google Play / Google Publisher API

当前交付分支中，不需要额外单独启动独立 UI 服务或独立 Gateway 服务。Dashboard 已由 Craftsman 统一提供。如查看仓库历史，可能会看到更早的 `dashboard/gateway`、`dashboard/ui` 目录；它们不属于本次交付基线。

3. 环境安装

推荐在仓库根目录使用一个统一虚拟环境，同时安装 Craftsman、Craftsman 的发布依赖和 Hunter。这样可以避免命令找不到、依赖分散和环境不一致的问题。

```
cd /opt/hunter-agent
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ./craftsman
python -m pip install -e "./craftsman[publish]"
python -m pip install -e ./hunter
```

安装后建议执行以下校验命令：

```
which python
which craftsman
which hunter
python -c "import langchain_core, google_play_scraper; print('dependencies ok')"
```

线上最低必需依赖是 Craftsman 主依赖、发布扩展依赖 `publish`、Hunter 主依赖。测试依赖如 `pytest`、`pytest-asyncio` 仅在验收或排障时再补充安装即可。

4. 关键配置项

建议通过 systemd `EnvironmentFile` 或安全密钥管理方式注入配置。开发环境可以使用本地 `.env` 文件。

文件/位置	关键配置项	说明
craftsman/.env 或 systemd 环境文件	DEEPSEEK_API_KEY, API_TOKEN, ANDROID_RELEASE_TRACK=internal, PACKAGE_POOL	Craftsman 服务端基础配置
craftsman/.env 或 systemd 环境文件	GOOGLE_PLAY_SERVICE_ACCOUNT_FILE 或 GOOGLE_PLAY_SERVICE_ACCOUNT_JSON	Google Play 发布认证
craftsman/.env 或 systemd 环境文件	ANDROID_BUILD_BACKEND, SKIP_GRADLE_BUILD, RELEASE_REQUIRE_HUMAN_APPROVAL	Android 构建与发布策略
hunter/.env 或 systemd 环境文件	DEEPSEEK_API_KEY, CRAFTSMAN_API_TOKEN, CRAFTSMAN_BASE_URL	Hunter 调用 Craftsman 所需配置
hunter/.env 或 systemd 环境文	TAVILY_API_KEY	可选网页搜索补充能力；

件		Google Play 发现仍以 Play 公开数据为主
---	--	------------------------------

配置注意事项：

1. `PACKAGE\_POOL` 中填写的是已经在 Play Console 预创建、并已授权给 service account 的包名。
2. `ANDROID\_RELEASE\_TRACK` 当前建议固定为 `internal`。
3. 真实发布时应确保 `SKIP\_GRADLE\_BUILD=false`。
4. 真实密钥、service account JSON、keystore、数据库和 workspace 不建议上传至代码仓库。

5. 服务启动与验证

推荐先启动 Craftsman，再触发 Hunter。Hunter 是触发器，Craftsman 是长期运行的服务端。部署时建议使用仓库根目录的共享虚拟环境，并在同一个环境中同时安装 Craftsman、Craftsman 发布依赖和 Hunter。

5.1 手动启动方式

```
cd /opt/hunter-agent
source .venv/bin/activate
python -m craftsman.cli serve --host 127.0.0.1 --port 8791
```

如果仅在 `craftsman/` 子目录单独创建虚拟环境，而没有把 `hunter` 也安装进同一个环境，后续 `hunter autopilot` 将无法正常运行。因此交付环境建议统一使用仓库根目录共享 `.venv`。

另开一个终端触发 Hunter：

```
cd /opt/hunter-agent
source .venv/bin/activate
hunter autopilot --publish --timeout 1800 --base-url http://127.0.0.1:8791
```

5.2 systemd 推荐方式

```
ExecStart=/opt/hunter-agent/.venv/bin/python -m craftsman.cli serve --host 127.0.0.1 --port 8791
Restart=always
RestartSec=5
```

仓库自带的 `docker/systemd/craftsman.service` 示例当前使用 `--host 0.0.0.0`。如果线上采用 Nginx 反向代理，建议在正式部署时改为 `127.0.0.1`，避免直接暴露服务端口。

5.3 启动后验证

`curl http://127.0.0.1:8791/health`
浏览器访问 `http://127.0.0.1:8791/dashboard`
只有在 `/health` 返回正常后，再触发 Hunter 自动流程。

6. 需求发现、代码生成与发布的实际流程

6.1 需求发现

Agent A 当前通过 `google-play-scraper` 读取 Google Play 公开搜索结果、应用详情和低分评论，形成真实的候选需求池。处理流程包括：关键词搜索、竞品扫描、评论采集、痛点聚类、候选评分和人工/自动选择。

6.2 代码生成

Agent B 会先生成实现计划，再产出 Android 工程、截图、图标和质量报告。质量报告用于判断当前生成结果是否达到可发布标准。质量分较低的结果应继续打磨，不建议直接进入发布。

6.3 自动发布

当使用 `hunter autopilot --publish` 时，系统会在需求发现、代码生成达标后，继续调用发布链路，自动完成包名分配、AAB 构建、签名和 Google Play internal 轨道上传。

Dashboard 的主要职责是观察和确认过程，不要求每一步都在页面手工触发。

7. 外网访问与端口说明

项目	默认值	建议
Craftsman 服务端口	8791	建议仅监听 127.0.0.1，由 Nginx 反向代理
对外访问端口	80 / 443	建议只开放 Nginx 端口
Hunter 入站端口	无	通常无需对外开放

如果需要直接通过公网 IP 访问 Craftsman，可将 `serve` 改为 `--host 0.0.0.0`，并同步在安全组、防火墙、访问控制和 HTTPS 层面做好保护。正式环境更推荐采用 Nginx + 443 的方式对外提供服务。

8. Google Play 真实发布前提

当前系统支持真实上传至 Google Play `internal` 测试轨道，但前提是目标 App 已经在 Play Console 中预创建。系统本身不负责稳定创建全新的 Play Console App 主体。

发布前建议逐项确认：

1. 已在 Play Console 创建目标 App，并确认包名。
2. 该包名已加入 `PACKAGE\_POOL`。
3. Google Play service account 已授权到对应 App。
4. release keystore 可用。
5. 隐私政策、商店文案、图标、截图已齐备。
6. Android 构建环境可用，且允许真实构建 AAB。

发布状态通常会经历：`prepared -> approved -> submitting -> building\_aab -> uploading\_internal -> internal\_submitted`。

当状态进入 `internal\_submitted` 后，可在 Play Console 对应 App 的 Internal testing 页面查看新的 release 版本、版本号、商店文案、截图和上传包体。

9. Android 构建与 Docker 说明

若使用 Docker 构建模式，Craftsman 会在需要时自动调用 `hunter-craftsman/android-builder` 镜像完成 Android 构建，不需要人工手动执行 `docker run`。容器执行完成后会自动退出并删除，镜像本身保留用于复用。

```
docker build -f docker/Dockerfile.android-ci -t hunter-craftsman/android-builder .
```

如果服务器不使用 Docker，则需要提前准备本地 Android SDK、Gradle 和签名环境。正式发布前，建议确认 `SKIP\_GRADLE\_BUILD=false`，避免只完成演示级流程而未真正产出可上传 AAB。

10. 数据目录、Workspace 与日志维护

生成过程中的工程文件、产物、截图、AAB、日志和数据库会保存在 workspace、数据库目录和服务日志中。失败任务产生的临时工程可以按运维策略定期清理，但建议先确认不再需要回溯。

日志建议通过 systemd `journalctl` 或统一日志平台管理，不建议在没有备份的情况下直接大范围删除。数据库、workspace 和发布产物目录建议纳入备份策略和容量监控。

11. HTTP 500 与常见问题排查

如果 Hunter 调用 Craftsman 时收到 `HTTP 500`，说明请求已到达服务端，但需要结合 Craftsman 运行日志确认具体原因。当前较常见的一类问题是 SQLite 版本兼容性。

```
journalctl -u craftsman -n 200 --no-pager
journalctl -u craftsman -f
curl http://127.0.0.1:8791/health
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

当前代码中仍有两处数据库写入使用 `RETURNING id`。如果运行环境 SQLite 版本过低，可能出现 `sqlite3.OperationalError: near "RETURNING": syntax error`。处理方式为：确认日志是否为该报错；随后将运行环境升级到 SQLite 3.35 及以上，或应用兼容性修补后重新安装并重启服务。

仅安装 `pysqlite3-binary` 并不能保证问题自动消失，关键仍是当前服务实际加载的 SQLite 运行时版本，以及服务是否已经在修改后重启。

12. 上线验收清单

检查项	验收标准
服务启动	`/health` 正常，Dashboard 可访问
环境安装	统一虚拟环境中同时存在 craftsman 和 hunter 命令
外网访问	通过 HTTPS 443 正常访问，不直接裸露内部端口
需求发现	可看到 discovery events、候选需求、证据和评分
代码生成	可生成工程、截图、质量报告和可追溯 run_id
Android 构建	可真实构建 APK/AAB，非演示模式
Google Play 发布	目标包名来自 PACKAGE_POOL，状态到达 internal_submitted
Play Console 验证	Internal testing 页面可看到新 release
运维保障	日志、数据库、workspace 已纳入备份与清理策略

13. 交付使用建议

建议甲方在正式环境中采用“先完成 Play Console 预创建与授权，再执行真实自动发布”的方式推进。这样可以把需求发现、代码生成、构建和 internal 轨道上传串成稳定闭环，同时保留 Dashboard 作为全流程观察与排障入口。

如需继续扩展到更高频率的自动发布、更多应用类型或更复杂的审核流程，建议在当前部署稳定后，再逐步增加监控、告警、构建容量和质量门槛策略。